FASE II

Manual de despliegue de Fase II
Asistente de preparación de revisiones de portafolio



Vértice AI Consulting · Convertimos datos en decisiones

Materia:	*Gestión de Proyectos de Inteligencia Artificial*
Actividad:	Fase II · Módulo 2 / Semana 8 · Temas 15 y 16
Alumno(a):	*Eduardo Aburto Angulo*
Matrícula:	*AL07182743*
Docente:	*M. en C. Luis Ariel Vázquez Piña*
Fecha de entrega:	30 - 10 - 2026

Declaración de Uso de Inteligencia Artificial

El alumno debe indicar de manera honesta si utilizó herramientas de Inteligencia Artificial (IA) para el desarrollo de esta actividad. Completa la siguiente tabla:

Herramienta de IA utilizada	¿La usaste?	Propósito / Descripción de uso
ChatGPT / OpenAI	Sí	Programación, pruebas, integración Docker y edición documental con ChatGPT/Codex. Evidencias procedentes de ejecuciones locales; cloud documentado como procedimiento preparado.
Claude (Anthropic)	No	No utilizado para esta fase.
Gemini (Google)	No	No utilizado para esta fase.
Copilot (Microsoft)	No	No utilizado para esta fase.
Otra (especifica):	No	No utilizado para esta fase.

Desarrollo de Fase II

Eduardo Aburto Angulo · 30 de octubre de 2026 · Versión 1.0.0

Propósito y decisión

Integro el modelo académico de la Actividad 6 en una aplicación que puedo revisar como banquero Sr. Streamlit recoge la noticia o el escenario y FastAPI responde mediante HTTP. Docker proporciona el mismo entorno para ambos componentes. La ejecución local está comprobada; el procedimiento cloud siguiente está preparado y no se presenta como una ejecución realizada.

Mi prioridad es preservar el perfil del cliente y revisar su beneficio neto. El clasificador sólo detecta noticias negativas en inglés; no estima rentabilidad ni recomienda un fondo. Los 150 casos de superhéroes y villanos son ficticios, con posiciones de 5 a 20 millones MXN, para trabajar sin datos personales.

Requisitos y componentes

Necesito Git, Docker Desktop abierto en Mac/Windows o Docker Engine con Compose en Linux. La referencia cloud es Ubuntu 24.04 LTS, 2 vCPU, 4 GB RAM y 20 GB de disco; son recursos iniciales propuestos, no una garantía de capacidad ni una cotización. No se necesita GPU. La construcción requiere salida a Docker Hub y al repositorio de paquetes Python.

El Dockerfile utiliza Python 3.12 y dependencias directas fijadas. El proceso corre como usuario 10001. Compose monta un filesystem de sólo lectura, permite temporales en /tmp y retira capacidades Linux. La API comprueba el SHA256 del modelo antes de cargarlo. SERVICE selecciona api o web; PORT permite adaptar el puerto a otros servicios de contenedores.

El navegador accede a Streamlit en 8501. Streamlit consulta http://api:8000 mediante la red de Compose y un encabezado X-API-Key. La API proporciona /health, /v1/model, /v1/predict, /v1/clients y /v1/scenario. Los puertos del host sólo escuchan en 127.0.0.1. El usuario no necesita conocer la clave para usar la pantalla local; por eso el acceso a la pantalla debe permanecer privado.

Ejecución local paso a paso

Desde una terminal, clono el repositorio y genero una clave aleatoria. Si ya existe .env, conservo su contenido o hago una copia antes de reemplazarlo. No subo ese archivo a GitHub.

```
git clone https://github.com/eduardoaburtoa/fase2-despliegue-portafolios.git
cd fase2-despliegue-portafolios
python3 -c "import secrets,pathlib;
pathlib.Path('.env').write_text('API_TOKEN='+secrets.token_urlsafe(32)+'\n')"
chmod 600 .env
docker compose config --quiet
docker compose up --build -d --wait --wait-timeout 180
docker compose ps
```

Espero que api y web aparezcan healthy. Abro http://localhost:8501, pulso Preparar revisión y compruebo la probabilidad 35.4 % del ejemplo. En Escenario del cliente elijo un perfil compatible y comparo los costos; un perfil distinto debe bloquear la elegibilidad. La documentación interactiva está en http://localhost:8000/docs; sus endpoints protegidos necesitan X-API-Key.

Para comprobar la red entre contenedores sin copiar credenciales:

```
docker compose exec -T web python -c "from frontend.client import call; print(call('/v1/model')
['sha256'])"
docker compose logs --tail 50
```

Para ejecutar la suite completa uso Python 3.12 en un entorno separado. Las instrucciones reproducibles están en README. La regresión HTTP recorre 452 noticias y guarda métricas; no reentrena ni selecciona otro umbral. Al terminar apago los servicios con docker compose down. Esto no borra los archivos fuente.

Preparación de Google Cloud

Propongo una máquina Compute Engine y un túnel SSH para una demostración privada. Antes de crear recursos reviso facturación, presupuesto y permisos de mi proyecto. Esta guía no implica que se haya contratado ni desplegado un servidor. Necesito la CLI gcloud autenticada y permisos para habilitar Compute, crear red, firewall y máquina. No asigno al contenedor permisos sobre servicios cloud.

Defino variables propias en mi equipo; sustituyo MI_PROYECTO y MI_IP_PUBLICA por valores reales. La IP permitida es la del equipo autorizado, no 0.0.0.0/0. Uso nombres nuevos para evitar modificar infraestructura existente.

```
export FASE2_PROJECT='MI_PROYECTO'
export FASE2_REGION='us-central1'
export FASE2_ZONE='us-central1-a'
export FASE2_CIDR='MI_IP_PUBLICA/32'
gcloud config set project "$FASE2_PROJECT"
gcloud services enable compute.googleapis.com
gcloud compute networks create vertice-fase2 --subnet-mode=custom
gcloud compute networks subnets create vertice-fase2-subnet \
  --network=vertice-fase2 --region="$FASE2_REGION" --range=10.20.0.0/24
gcloud compute firewall-rules create vertice-fase2-ssh \
  --network=vertice-fase2 --allow=tcp:22 \
  --source-ranges="$FASE2_CIDR" --target-tags=vertice-fase2-ssh
gcloud compute instances create vertice-fase2 \
  --zone="$FASE2_ZONE" --machine-type=e2-medium \
  --subnet=vertice-fase2-subnet --tags=vertice-fase2-ssh \
  --image-family=ubuntu-2404-lts-amd64 --image-project=ubuntu-os-cloud \
  --boot-disk-size=20GB --no-service-account --no-scopes
gcloud compute ssh vertice-fase2 --zone="$FASE2_ZONE"
```

No abro 8000 ni 8501 en el firewall cloud. Si una política corporativa impide IP externa o SSH directo, uso el mecanismo de acceso aprobado por el administrador, por ejemplo IAP, y adapto la red antes de continuar.

Instalación en la máquina y acceso privado

Dentro de la máquina Ubuntu instalo Docker siguiendo el repositorio apt oficial. El script scripts/install_docker_ubuntu.sh documenta las mismas operaciones; lo reviso y lo ejecuto únicamente en Ubuntu 24.04 preparado para esta demostración. No fue ejecutado en un servidor cloud durante esta entrega.

```
sudo apt-get update
sudo apt-get install -y git python3 ca-certificates curl
git clone https://github.com/eduardoaburtoa/fase2-despliegue-portafolios.git
cd fase2-despliegue-portafolios
sudo bash scripts/install_docker_ubuntu.sh
python3 -c "import secrets,pathlib;
pathlib.Path('.env').write_text('API_TOKEN='+secrets.token_urlsafe(32)+'\n')"
chmod 600 .env
sudo docker compose up --build -d --wait --wait-timeout 180
sudo docker compose ps
sudo docker compose exec -T web python -c "from frontend.client import call;
print(call('/v1/model')['sha256'])"
```

En otra terminal de mi equipo abro el túnel. Si Docker local sigue usando los puertos, lo detengo primero o uso puertos locales alternativos 18501 y 18000.

```
gcloud compute ssh vertice-fase2 --zone="$FASE2_ZONE" -- \
  -N -L 18501:127.0.0.1:8501 -L 18000:127.0.0.1:8000
```

Abro http://localhost:18501 y http://localhost:18000/docs. Repito noticia, escenario, credencial inválida y salud. Guardo la revisión Git, ID de imagen, arquitectura y fecha, junto con capturas. Una construcción ARM64 local no acredita por sí sola una ejecución AMD64 cloud; ese paso debe comprobarse en la máquina de destino.

Operación y reversión

Registro git rev-parse HEAD y docker image inspect vertice-fase2:1.0.0. Antes de una actualización guardo la imagen validada con docker tag vertice-fase2:1.0.0 vertice-fase2:respaldo y conservo el commit anterior. Si falla la nueva versión, recupero el commit anterior, restauro la etiqueta desde respaldo y uso docker compose up -d --no-build --pull never. No elimino el respaldo hasta aceptar la actualización.

Reviso salud, errores y consumo con docker compose ps, logs y docker stats. No registro textos completos, claves ni datos personales. Para rotar la clave actualizo .env y recreo ambos servicios con docker compose up -d --force-recreate. Antes de un piloto real faltan autenticación individual, TLS para exposición web, límites de solicitudes, gestión de secretos, pruebas de carga y validación externa con datos aptos para ese uso.

Si falla la construcción, reviso conexión y espacio; si falta el token, reviso .env; si la API no está healthy, consulto sus logs e integridad del modelo. Un 401 indica clave inválida y un 422 datos no válidos o texto sin vocabulario conocido. No resuelvo un error de integridad omitiendo el control.

Detener una VM no elimina cargos de disco. Tras la demostración decido si conservo o elimino los recursos desde Google Cloud, verificando antes qué información se perdería. No se ha ejecutado esa eliminación como parte de esta entrega.

Fuentes y asistencia documental

Docker: https://docs.docker.com/compose/how-tos/startup-order/ y https://docs.docker.com/engine/install/ubuntu/

Google Cloud: https://docs.cloud.google.com/compute/docs/instances/create-start-instance y https://docs.cloud.google.com/compute/docs/connect/standard-ssh

El código conserva el modelo de https://github.com/eduardoaburtoa/actividad6-evaluacion-portafolios . La documentación y las pruebas se elaboraron con apoyo de ChatGPT/Codex; no atribuyo a Copilot un uso que no realicé. Distingo la ejecución local verificada del procedimiento cloud preparado.